

Code Challenge

Primary Objective

Setup a web api project with SQLite as database.

Required APIs

Post : host/api/product

request

```
{
    name: "",
    category: grocery/fashion
    amount: 00.00,
    description: "",
    baseDiscountInPercentage : 00.00
}
```

Put : host/api/product

request

```
{
    id: 22,
    name: "",
    amount: 00.00,
    description: "",
    category: grocery/fashion
    baseDiscountInPercentage : 00.00
}
```

Get: host/api/products/{product_id}

response

```
[
    {
        id: 22,
        name: "",
        amount: 00.00,
        description: "",
```

```
        category: grocery/fashion
        baseDiscountInPercentage : 00.00
    }
]
```

Post : host/api/order

request

```
{
    customerphone: ####,
    items:
    [
        {
            productId: 11
            quantity: 1
        }
    ],
    note: ""
}
```

Put : host/api/order

request

```
{
    orderId: 22,
    customerphone: ####,
    note: "sdfsd"
}
```

Post : host/api/order/{orderId}/item

request

```
{
    productId: 11
    quantity: 1
}
```

```
}
```

Get: host/api/order/{orderId}

response

```
[
```

```
{
```

```
    orderId: 22,
```

```
    customerPhone: #####,
```

```
    note: "sdfsd"
```

```
    createdOnDatetime,
```

```
    status: pending/ processed,
```

```
    items:[
```

```
        {
```

```
            productid: 2,
```

```
            productname: 'jeans',
```

```
            quantity:2 ,
```

```
            netAmount(after application of discount): 200.00,
```

```
            actualAmount: 250.00
```

```
        },
```

```
        {
```

```
            productid: 3,
```

```
            productname: 'milk',
```

```
            quantity:3 ,
```

```
            netAmount(after application of discount): 50.00,
```

```
            actualAmount: 62.00
```

```
    ]},
```

```
    actualTotalPrice: 312.00,
```

```
    netTotalPrice(after application of discount): 250.00
```

```
]
```

Delete : host/api/order/{orderId}

Delete : host/api/order/item/{itemId}

Post : host/api/order/{orderId}/process

The above APIs should insert/update/delete data from the corresponding tables

Note: The data given along the contract is for reference only.

Process API Logic

Once the process API is invoked

- The discount logic given below should be applied on the respective order and is to be saved on the db.
- The status of the respective order should change from 'pending' to 'processed'.

Discount Calculation Logic

- For grocery
If the quantity is more than 2, then 10% discount should be applied
If the quantity is 2, then 5% discount should be applied
If the quantity is 1, base discount if any should apply
- For fashion
If a total of 3 or more fashion products are ordered, then the discount on those products should be 15%.
If less than 3 items are ordered, then base discount should be applied

Eg 2 shirts and 1 jeans are ordered, then the 15% discount would be applicable on the price of these products. 15% on the price of shirts and 15% on the price of jeans.

If case of just 1 shirt and 1 jeans, their respective base discount should be applied.

The maximum total discount for an order should be limited to 500rs. In this case discount can be distributed equally to the respective products.

Suggestions

- Use entity framework core.
- Implement Domain pattern.
What we mean by domain
In our case, Product, Order must be considered as domains, any logic should be enclosed within those objects instead of writing it in separate service classes. You should apply possible OOPs concepts on these objects.

Evaluation Criteria

- All the functionalities asked on the task should work bug free.
- Validation checks would be evaluated.
- Usage of good code practices
- On time submission. Early submission would be given brownie points.